

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: df = pd.read_csv("Advertising.csv")
dfs = df.iloc[:20,:]
dfs
```

```
Out[2]:
```

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9
5	6	8.7	48.9	75.0	7.2
6	7	57.5	32.8	23.5	11.8
7	8	120.2	19.6	11.6	13.2
8	9	8.6	2.1	1.0	4.8
9	10	199.8	2.6	21.2	10.6
10	11	66.1	5.8	24.2	8.6
11	12	214.7	24.0	4.0	17.4
12	13	23.8	35.1	65.9	9.2
13	14	97.5	7.6	7.2	9.7
14	15	204.1	32.9	46.0	19.0
15	16	195.4	47.7	52.9	22.4
16	17	67.8	36.6	114.0	12.5
17	18	281.4	39.6	55.8	24.4
18	19	69.2	20.5	18.3	11.3
19	20	147.3	23.9	19.1	14.6

```
In [3]: tv = dfs.iloc[:,1]
radio = dfs.iloc[:,2]
news = dfs.iloc[:,3]
sales = dfs.iloc[:,4]
```

```
In [4]: # Standardize data by z-score
tv_standard = (tv - tv.mean()) / tv.std()
```

```
radio_standard = (radio - radio.mean()) / radio.std()
news_standard = (news - news.mean()) / news.std()
sales_standard = (sales - sales.mean()) / sales.std()
```

```
In [5]: # Objective function is the mean squared error
# f(m, b) = (1/n) * (sum (( m * x[i] + b) - y[i]) ** 2 )
# Substitute standardized data into mean squared error
def f(m1, m2, m3, b):
    sales_pred = m1 * tv_standard + m2 * radio_standard + m3 * news_standard + b
    return ((sales_pred - sales_standard) ** 2).mean()

print(f(0, 0, 0, 0))
```

0.9500000000000002

```
In [6]: # Gradient: Partial derivative
def f1(m1, m2, m3, b):

    sales_pred = m1 * tv_standard + m2 * radio_standard + m3 * news_standard + b
    error = sales_pred - sales_standard

    # Partial derivatives
    grad_m1 = 2 * (error * tv_standard).mean()
    grad_m2 = 2 * (error * radio_standard).mean()
    grad_m3 = 2 * (error * news_standard).mean()
    grad_b = 2 * error.mean()

    return grad_m1, grad_m2, grad_m3, grad_b

f1(0, 0, 0, 0)
```

```
Out[6]: (np.float64(-1.6395571500810813),
np.float64(-0.8162598518330719),
np.float64(-0.3886107126867289),
np.float64(4.4131365228849973e-16))
```

```
In [7]: # Gradient descent
m10, m20, m30, b0 = 0, 0, 0, 0
alpha = 0.1

for i in range(5000):
    gradient_m1, gradient_m2, gradient_m3, gradient_b = f1(m10, m20, m30, b0) # par

    m10 = m10 - alpha * gradient_m1
    m20 = m20 - alpha * gradient_m2
    m30 = m30 - alpha * gradient_m3
    b0 = b0 - alpha * gradient_b

    if i % 500 == 0:
        print("Iteration {i} - Mean squared error: ", f(m10, m20, m30, b0))
```

```

Iteration {i} - Mean squared error: 0.6364814027268598
Iteration {i} - Mean squared error: 0.06852898782784961
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964
Iteration {i} - Mean squared error: 0.06852898782784964

```

```

In [8]: # Reverse engineer to get true values of m1, m2, m3, b
m1 = (m10 * sales.std()) / tv.std()
m2 = (m20 * sales.std()) / radio.std()
m3 = (m30 * sales.std()) / news.std()
b = sales.mean() + sales.std() * b0 - ((m10 * sales.std() * tv.mean()) / tv.std())

print("m1: ", m1,
      "\nm2: ", m2,
      "\nm3: ", m3,
      "\nb: ", b)

```

```

m1: 0.055244099951379994
m2: 0.16885567654598685
m3: -0.015225252984652996
b: 2.8593828453004493

```

```

In [9]: # Check with minimize function
from scipy.optimize import minimize

# Objective function: Mean squared error
def fMin(w):
    m1, m2, m3, b = w
    sales_pred = (m1 * tv) + (m2 * radio) + (m3 * news) + b
    return ((sales_pred - sales) ** 2).mean()

res = minimize(fMin, x0 = [0, 0, 0, 0])
res.x

```

```

Out[9]: array([ 0.05524408,  0.16885569, -0.01522528,  2.85938665])

```